

2/4

Team finished needs statements

I wrote about half the requirements

I brought up usage of a gyroscope sensor to detect which cameras are down to reduce dark sent images

2/9

Clarified project scope (no changes to external structure, modify connections between camera and pi, ideally use one pi) with professor and contents of requirements and report (contain both reflection of the old material and some new material focusing on hardware changes)

Made Gantt chart and budget:



Item	Quantity	Unit Cost (USD)	Total (USD)
Raspberry Pi 4 Model B, 1GB	1	38.50	38.50
Raspberry Pi Camera Module 3 Wide	3	37	111
Anker Power Bank 5V/3A	1	20	20
Multi Camera Adapter Module	1	50	50
3D-printer filament (ABS, 1 kg)	1	11	11
Solder	-	5	5
Wire spool (22–24 AWG)	1	3	3
Misc. fasteners, heat-shrink, etc.	-	~5	~5
Estimated total			243.50

2/11

Presented. The presentation lacks a lot of detail, especially on the proposed design.

Working on improving design slides: 5 slides, one usage example + block diagram, one hardware overview, one software overview, one 3D mapping process, one camera multiplexing, one object detection

Reviewing codebase to learn about how the current state works

- Camera clients on RPi's are set up with a fps. Not sure why 1fps is the default, seems very slow and the pi and camera are capable of capturing and sending faster I think. Maybe to reduce server load. This is an important bottleneck to understand
- The report mentions that orientation is found by evaluating the images. Not found in the camera client, so it must be done by the server, so it won't be useful in turning on/off cameras on the board natively when multiplexing

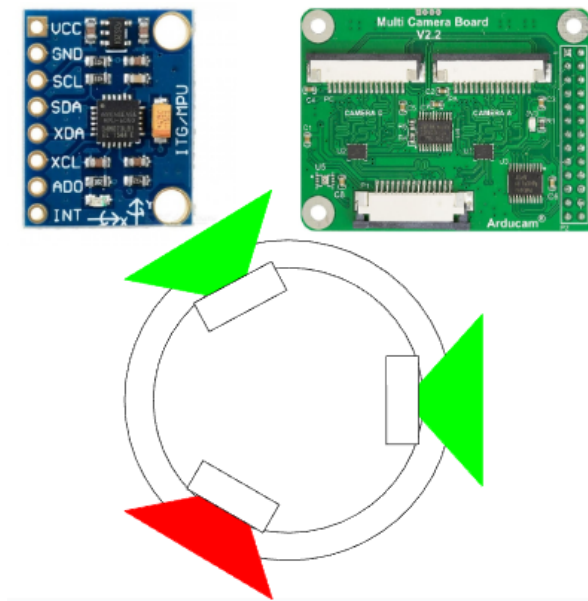
From my research on what we need for multiplexing cameras:

I think adding a gyroscope is a good idea because it will let us know orientation. accelerometers aren't reliable at the speed our ball moves at and 9 degrees of freedom IMU (includes magnetometer in addition to gyro and accelerometer) is overkill. Using computer vision probably isn't useful here because that would require the server to identify the floor instead of the pi, creating more back and forth and requiring the pi to capture with all cameras to identify the downward camera.

the only feasible alternative I can think of asking the pi to take average brightness across all 3 images and using the darkest as floor, but that has three downsides: all 3 cameras have to be on to get a comparison, dark objects may be counted as the floor, and the rpi might not have enough ram (1GB) to open the images efficiently.

MPU 6050 is a good choice of gyroscope

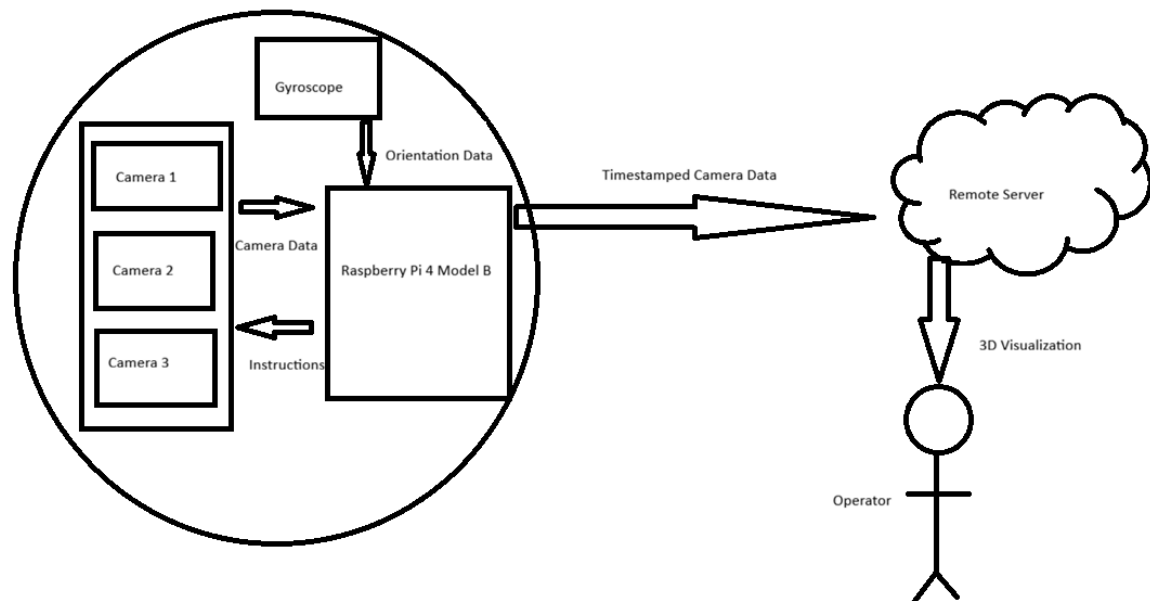
MPU 6050 (top left), proposed 3 to 1 camera multiplexing board (top right), downward facing camera visualization (bottom)



Done: Updated needs statement to have more statistics, added one societal impact via low cost -> scalable impact, added gyroscope to budget, filled in camera multiplexing slide with diagram

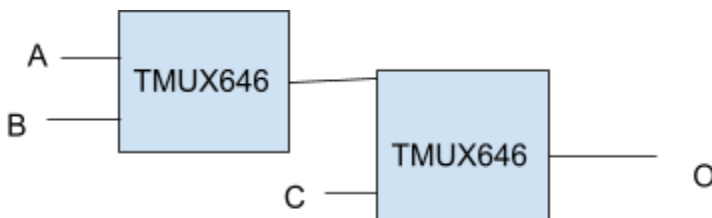
2/13

Made system diagram:



Doing work today on the report led to a lot of datasheet looking and thinking about multiplexing strategy:

Getting more and more concerned about the nature of the camera multiplexing. The board to multiplex the cameras is not official from Raspberry Pi and could be a weak point. Also, it is expensive and is not a parallel multiplexer anyways, and can only take images in sequence. I was looking at cascading 2 2:1 CSI muxs (made by TI) to create a 3:1 mux to do the same thing, but chatgpt says that would likely not work. Although, it would be so cheap (~\$2) that I think it is still worth trying.

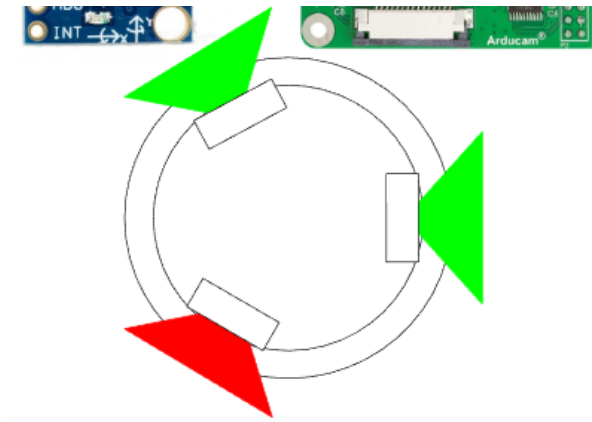


For our requirements, we do not need to use CSI cameras; they are fast, but we don't need fast. Realistically, we might capture up to 10 fps. USB cameras would be just fine to do this. There are 2 USB 3 and USB 2 ports each. We could completely get rid of the need for a multiplexer if it works. However, it requires more CAD and will put a lot more load on the Pi's CPU (The pi

camera module has some hardware acceleration or something), but the software changes would likely be minor.

It might be best to check asap if the Arducam multiplexer works in case we need to pivot. Will bring up this at Monday meeting ^

Realized all three cameras could be capturing valuable data if they are all along the hemisphere: imagine a top down view where all cameras point outwards:



2/16

Focus on fire environment application

3 major changes to evaluate:

- IR cameras for use in smoke
- RPi 4 vs 5, which has minimum spec needed
- Define how gyro will interface with the multiplexing logic
 - Gyroscope tracks orientation of ball (which way is facing down)
 - Define a dead zone spherical cap at the bottom of the ball
 - Raspberry pi knows the relative position of cameras
 - If a camera enters the dead zone, cut it

2/18

RPi 4 vs 5

- 5 is more powerful but requires 5V/5A, power banks don't natively support it (need converter)
- RPi 4 can still handle the workload

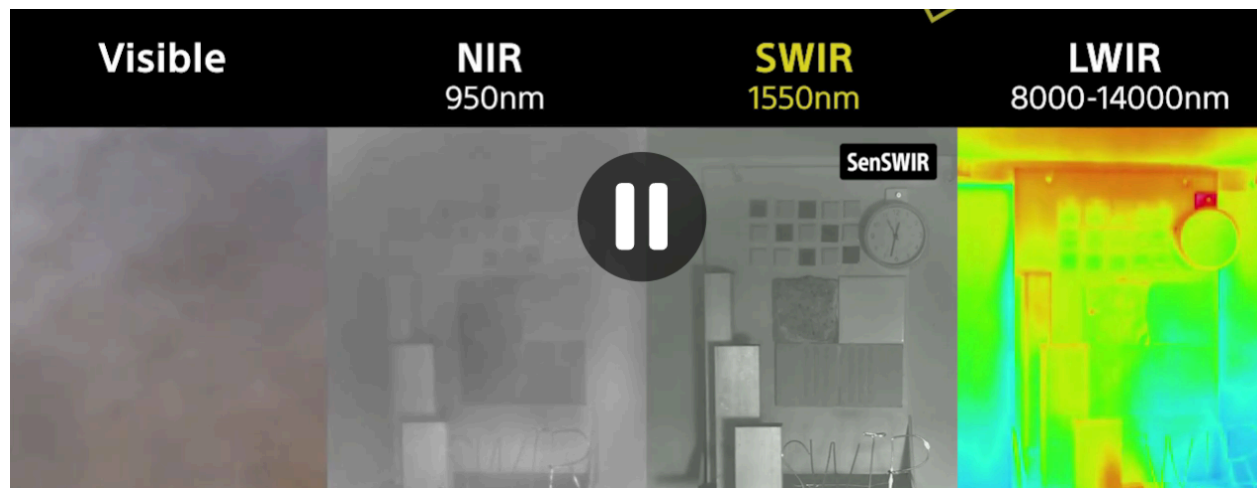
Cameras

- IR for smoke application

RPi RAM needed

Buying NoIR, but it will probably struggle to see through smoke. Cannot find any cameras by RPi can properly see through smoke

NoIR will see Visible and NIR, but need specialized camera to see SWIR or LWIR



2/23

Will need to use Bullseye(libcamera/rpicam). Aducam camera adapter only supports Bookworm(rpicam) and Bullseye(libcamera/rpicam), but does not support IMX708 on Bookworm. The rpi cameras we use are both IMX708.

2/27

GPIO pin conflicts. Peripherals (camera multiplexer and gyroscope) might use the same gpio pins.

Three options, stacking headers, USB adapter from STEMMA QT (if using gyro with this port), or use bitbanging

Stacking headers: pins need to be safe to have multiple connection. Gyro minimally requires VIN, GND, SDA, SCL.

Adapter: requires hardware, but is easy

Bitbanging: slower, harder, no hardware <https://forums.raspberrypi.com/viewtopic.php?t=173272>

3/2

Feasibility of using gyroscope for orientation detection:

Maximum Angular rate measurement range: ± 2000 deg/s

Diameter of ball: 6in

Circumference: 18.84 in = 0.47 m

Desired:

Maximum speed: 5 m/s

Maximum revolutions per second: $5/0.47 = 10.638$

Maximum deg per second: ~ 3600

Maximum using 2000 deg/second gyroscope:

Maximum revolutions per second: $2000/360 = 5.556$

Maximum speed: $5.556 * 0.47 = 2.61 \text{ m/s}$

2.6 m/s is pretty slow, less than 6 mph. It is possible that the ball may slide somewhat and slightly decrease the revolutions per distance, but not reliable. Is it still worth it to include the gyroscope?

Pros:

- Used to save camera usage
- Does not require camera data to determine downward camera
- Does not require cloud server to determine downward camera
- Can simply switch off when not usable (detect when saturated)

Cons:

- Ball will often travel faster than 2.6 m/s, at which point it is not useful
- Could be difficult to connect in addition to multiplexer (see 2/27)

3/4

Comparing gyroscopes that fit 4000 deg/s spec

Option A: <https://www.mikroe.com/gyro-5-click>

<https://download.mikroe.com/documents/datasheets/PS-ITG-3701.pdf>

Option B: <https://www.adafruit.com/product/4502>

<https://www.st.com/resource/en/datasheet/ism330dhcx.pdf>

Both support both I2C or SPI

Both have 4000 deg/s

“A” is only gyroscope (no accelerometer), has 8.2 LSB/deg/s sensitivity at 4000deg/s

“B” has accelerometer as well as gyro, has 140 mdeg/s/LSB sens at 4000deg/s

(7.14LSB/deg/s), and is sold by major retailer that has extensive software

A is slightly more performant, and we don't necessarily need an accelerometer. B is \$5 cheaper and faster shipping with more software support.

Prefer B for convenience and well documented libraries

3/22

Calculating orientation:

Adafruit gyroscope does not natively calculate orientation, only provides xyz acceleration and angular velocity.

Angular velocity can be summed over time to come up with angular orientation (will cause drift over time)

Pseudocode without gyroscope bias correction ([API reference](#)):

```
gyro_range = 2000 deg/s
```

```
# operator should let ball rest on the ground
```

```
reset() # reset sensor's configuration
```

```
angle_x, angle_y, angle_z = 0
```

```
last_time = now()
```

```
while True:
```

```
    x, y, z = imu.gyro # The x, y, z angular velocity values in a 3-tuple in radians / second
```

```
    now = now()
```

```
    dt = now - last_time
```

```
    angle_x += dt * x
```

```
    angle_y += dt * y
```

```
    angle_z += dt * z
```

```
    vector = <angle_x, angle_y, angle_z>
```

```
    u = vector / magnitude(vector)
```

```
    comparison = dot_product(u, <0, 0, -1>) # the downward vector
```

```
    if comparison < threshold: cut camera
```

```
    last_time = now
```

If this does not work because there is too much drift, then there is little other option. Cannot use accelerometer data without magnetometer data for rolling objects (no idea where gravity is without it). Otherwise we could try to have either the Pi or the server take a look at captured images and predict when to turn off cameras, but that is high latency.

3/29

Bitbanging instructions for our project, developed from a few sources:

Need to create new SDA/SCL pins that the HAT covers

- Identify existing buses using `i2cdetect -l`
- Add the following line to `/boot/config.txt` and reboot
 - `dtoverlay=i2c-gpio,i2c_gpio_sda=18,i2c_gpio_scl=19,i2c_gpio_delay_us=5`
 - Replace 18 and 19 with the appropriate BCM pin names (pinout.xyz)

Need `busio.I2C` object and (optionally) address of target on the bus

- Run `i2cdetect -l` again to find new bus number
- After connecting the component to the new SDA/SCL lines, find the target's address using `i2cdetect -y <bus number>`
- Create `busio.I2C` object by passing the pins to it (use `board.D{BCM number}` to refer to the pins)
- Pass the object and the address number to Adafruit object constructor

The IMU and three cameras will always be in the same place on the ball. However, the ball can be in any orientation. How do we know whether the cameras are facing the ground? Say we are in a known orientation. Of course, through measurement, we can find out and remember where all the cameras are. Now say that we are in a different, unknown orientation. While still, the accelerometer will measure the acceleration due to gravity (yes, even while still, it measures the acceleration). If we store the expected acceleration in the known state, we can compare it to the new state, and this difference will tell us the angle with respect to the gravity vector. but this does not tell us the orientation of the cameras. For example, say our known state has gravity along the y component of the accelerometer. If we spin the ball on the ground, and then stop it, the accelerometer still has gravity along the same axis. However, at least two of the cameras have changed orientation. Therefore, it is necessary to have at least one more directional reference to initialize position.

Consider a base for the ball to rest on in an initial position that is always the same. Such a base could be made by having a flat surface with an asymmetrical prong, and having a matching hole on the ball's surface. Because the prong/hole are asymmetrical, there is only one way to place the ball onto the prong. This base can guarantee a single known orientation. This can be combined with the earlier accelerometer-based approach to determine the direction of gravity. Since we know the angle relative gravity, we can find the position of the ball.

Now we need to determine whether this is "worthwhile". A base would require additional time to deploy.

4/5 - IMU Connectivity Changes

Resorting to 3 RPi boards means we need to update how we communicate IMU data

Options

1. Connect multiple I2C masters (3 RPi boards) to single target (IMU)
2. Connect one target to RPi 4B, connect RPi 4B to other Pi's over GPIO

Option 2 is better, connecting multiple masters to a single target requires multiplexing and control-sharing logic. Also, the only communication needed is a flag (hi/lo) which is what GPIO does

4/11 - Cloud setup

1. Create a g2-standard-4 instance with >64 GB of storage in an area with high availability (eg east4)
 - a. Use Version Ubuntu 22.04 LTS Minimal x86/64, for best compatibility. Do not use Deep Learning OS, it has the wrong CUDA version
2. Follow instructions at <https://docs.nvidia.com/cuda/cuda-installation-guide-linux/index.html>
 - a. Install gcc: `sudo apt install gcc g++`
 - b. When instructed to install the cuda toolkit, use https://developer.nvidia.com/cuda-12-4-1-download-archive?target_os=Linux&target_arch=x86_64&Distribution=Ubuntu&target_version=22.04&target_type=deb_local
 - c. As in the post installation instructions, you need to export the path. Consult these instructions to add them to `.bashrc`, avoiding the need to do this every time you open a terminal
<https://askubuntu.com/questions/885610/nvcc-version-command-says-nvcc-is-not-installed>
3. Reboot
4. Install miniconda
 - a. `wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh`
 - b. `bash Miniconda3-latest-Linux-x86_64.sh`
 - i. do NOT use `sudo`, that would install for root user
5. Install git
 - a. `sudo apt-get install git`
6. Follow README instructions in the repo
<https://github.com/TeamBallers/MASt3R-SLAM-API/blob/main/README.md>
 - a. Notice that you need to use a mkl version earlier than 2025. Otherwise, you may experience a conflict in some places
 - b. ERROR: moderngl in in3d requires numpy >= 1.16 and < 2, but opencv in mast3r requires >= 2. Not sure what this breaks
7. Install tailscale
 - a. `curl -fsSL https://tailscale.com/install.sh | sh`
 - b. `sudo tailscale up (auth to tamu tailnet)`

Note: the existing script does not reset the incoming image directories, you need to reset them between scenes. Especially between switching camera types (eg iPhone vs RPi Camera Module 3)

4/14 - revisiting IMU camera cutting algorithm

Since the camera capture is blocking, we need to thread the imu reading. Then, we have a question of which prediction to use of the many that happen during the camera capture. Two options come to mind:

1. Estimate at what time the capture happens, use the prediction at that time

2. Find what percentage of predictions are downward, and determine downward orientation based on whether it passes a threshold

After implementing both and testing with various values/thresholds, the algorithm is most accurate with 55 degrees and 60% threshold

4/17

Taking images without specifying exposure time leads to significant motion blur due to fast rotation. At 10 rev/s, we get 20π rad/s. If we are aiming for less than 5% blur with our 120 degree ($2\pi/3$ rad) FOV, then we get $20\pi * t < 0.05 * 2\pi/3$. Solving for t gives 1.7ms or 1700 us.

That is a little low, so we need to constrain users to rolling the ball much slower. At 1 m/s, the ball still will pass through an average room, taken to be 6 meters, in 6 seconds, which is not very long compared to the time it takes mast3r-slam to run. Ball circumference is 0.55m. The ball makes around 2 revolutions per second, or 720 deg/s or 4π rad/s. Then, we can get $5t$ for our exposure time, or ~8ms. Given that rpi cameras typically operate at 0.5-10ms, this is acceptable. We can tell users to operate within these bounds.

Compared to experiment:

4 ms is good enough at the speed that users roll the ball. Motion blur still occurs, but reducing the exposure time further does not capture enough light in the use case (smoky/dark environments).

4/19

benchmark time to take picture, benchmark time to take picture by size

Images taken at higher resolutions have a larger field of view. An RPi engineer on the forums says that the Sony cameras they use are only capable of doing a two bit binning, so once resolution gets low enough they must crop.

We are motivated to capture images that create the best model with the fewest number of images. However, since the most useful images we take occur while the ball is rolling, taking large images with maximum fov will significantly decrease the number of images we capture, so we may not be able to build a satisfactory model. On the other hand, taking many small images may gather more useful data, but it is important to remember that in the end to end system, the model creation will take much longer with more images, as all images are downsized to the same resolution.

We already know that a lot of time will be spent processing images. Mast3r slam needs at least 30+ images to run effectively.

Then, we should test how well we perform taking images at the lowest resolution that fits each fov:

Resolution	Image
512x288 (desired size by mast3r)	
1024x576	
1536x864	
2304x1296	
4608x2592 (maximum sensor is capable of)	

It seems as there are only two fovs with keeping the rest of the configuration constant. The minimum resolutions for the narrow and wide angle fovs are 512x288 and 2304x1296 respectively. Then, we only have to test two different fovs.

Considering we have the choice of downsizing the high resolution image, we should consider whether it can really yield a faster upload speed. If downsizing before mast3r slam runs is faster

than passing the large image to mast3r slam, the entire process will definitely be faster since the pre-sized image does not have to be downsized by mast3r.

For simplicity, we start with a simple stride approach (take every $2304/512 = 4.5$ th pixel).

Image capture is known to take much less time than upload. Then, we should use some parallelism by continuing to take pictures and buffer them as we send them to the server. The ball can continue to upload the images after the ball has stopped rolling, but we want the cameras to always be capturing while still rolling in the center of the room.

Since we are using some kind of parallelism, we should measure the time it takes for the entire use case, not just one image.

Since it is faster, we can take the proper approach and try the same downsizing mast3r uses: lanczos resampling.

Approach	Upload iteration duration (s)	Jpeg bytes (B)	Images received after 10 seconds (one camera active)	Images received after 20 seconds (one camera active)	30 seconds	Maximum fire and forget submission time (s)
Upload 2304x1296	0.15	~5.9e5	3	8	15	2e-4
Lanczos downsize 2304x1296 to 512x288, upload 512x288	0.62	~1.73e4	6	25	38	1e-4

Although the iterations take less time (no need to downsize during the iteration), the time for the full image to completely travel through the pipeline is very slow due to the large size of the files. The image size ratio of without to with downsizing is about 34x. This results in many more images being uploaded. Also, because images are sent sequentially and the images are stuck in the pipeline, we do not see later images until all previous images have been uploaded. This is a problem for the large images because they take much longer to receive than they take to upload; we only see images of the ball's beginning. The downsized images solve this problem because they are emptied from the pipeline rapidly.

After startup, we get ~6 fps without downsizing and ~16 fps with downsizing.

4/24 - GUI improvements

Operator should only need to use GUI, no code

GUI should be able to trigger the ball to start capture, the easiest way is API. API exposes two endpoints, start and stop.

Experiment: Latency for ball to receive request from GUI ~200 ms. Latency to start camera capture, ~6 seconds.

Need to separate start/stop requests from camera initialization, which takes up the most time. If separated, the delay from request to first capture would be <1s